



# A Heuristic for Periodic Memory Allocation with Little Fragmentation to Train Neural Networks

Akifumi Imanishi

Preferred Networks  
Tokyo, Japan  
imanishi@preferred.jp

Zijian Xu

Preferred Networks  
Tokyo, Japan  
joe@preferred.jp

## Abstract

Neural network training requires immense GPU memory, and memory optimization methods such as recomputation are being actively researched. Recent improvements in recomputation have reduced more than 90 % of the peak allocated size. However, because it produces complex irregular allocation patterns, PyTorch's default caching allocator wastes up to 20 % of memory because of severe fragmentation and increases the cache management overhead. The periodic allocation patterns during training make offline memory optimization possible. Dynamic storage allocation (DSA) is the problem of minimizing the memory address ranges given the allocation pattern. This is defined as the 2D bin-packing problem, in which each rectangle can move only vertically. Although the first-fit or best-fit heuristics perform well for DSA, we propose a simulated annealing-based non-trivial heuristic algorithm that optimizes the *topological* ordering of allocations to further minimize fragmentation. The proposed algorithm evaluates a candidate allocation plan with  $O(\log N)$  amortized time, where  $N$  is the number of allocations. We empirically tested our algorithm on both randomly generated data and allocation patterns obtained by training popular vision and text models with recomputation. The experiments showed that, on average, our algorithm reduced fragmentation caused by the PyTorch caching allocator from 29.5 % to 0.4 %, compared to 5.3 % by the first-fit method.

**CCS Concepts:** • Mathematics of computing → Simulated annealing.

**Keywords:** Dynamic Storage Allocation, Fragmentation, Heuristics, GPU Memory

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org). ISMM '24, June 25, 2024, Copenhagen, Denmark

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0615-8/24/06

<https://doi.org/10.1145/3652024.3665508>

## ACM Reference Format:

Akifumi Imanishi and Zijian Xu. 2024. A Heuristic for Periodic Memory Allocation with Little Fragmentation to Train Neural Networks. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on Memory Management (ISMM '24)*, June 25, 2024, Copenhagen, Denmark. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3652024.3665508>

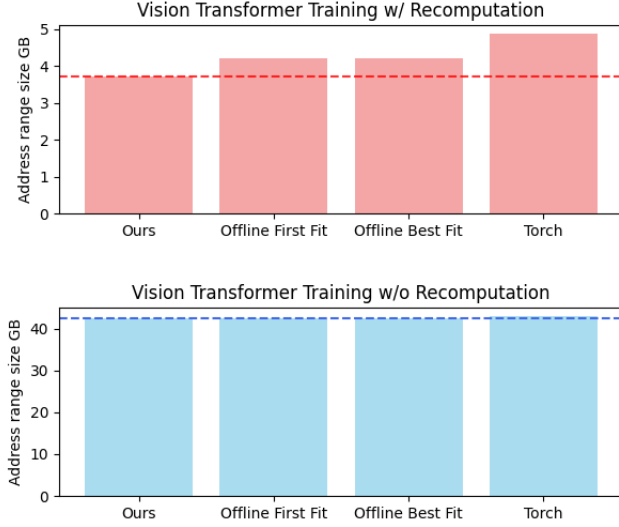
## 1 Introduction

### 1.1 Recomputation and Memory Fragmentation

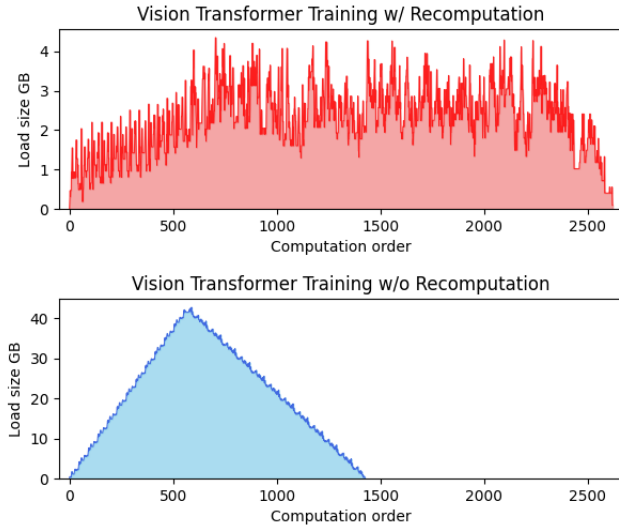
The training of modern neural networks is heavily constrained by GPU memory. As such, memory optimization has become a crucial topic of research. A multitude of methods have been proposed to reduce memory usage during training, including quantization [4], offloading [1], and recomputation [5]. Among these techniques, recomputation has shown promising results because it maintains computational accuracy while keeping the computation homogeneous. The principle behind recomputation is simple: Rather than retaining large tensors in memory for extended periods, it stores smaller inputs and only re-computes the large tensors as needed. Given a computational graph, recomputation solvers determine which values need to be recomputed and output a new computational graph. This new graph computes exactly the same results with a smaller memory footprint. The state-of-the-art recomputation solver can reduce the memory usage for the training of popular workloads by nearly 90 %, with a trade-off of an additional 30 % computational effort [12, 13].

Despite the optimization at the computation graph level by recomputation, there remains a significant discrepancy between the peak allocation size and actual runtime memory usage. As displayed in Figure 1, the peak allocation size for training the vision transformer (vit\_small\_patch16\_224) can be reduced more than ten-fold through recomputation. However, the PyTorch allocator still consumes 130.6 % of the peak allocation size post-recomputation. This demonstrates that approximately 23.4 % of the memory used is owing to allocator fragmentation, with these regions never actually used.

To better understand why recomputation leads to fragmentation, we have plotted the memory consumption and computation order, meaning the running progress of each operator, in Figure 2.



**Figure 1.** Intensive recomputation increases fragmentation. Each bar shows the size of address ranges used by the allocation algorithm and the dashed line stands for the peak load size. For each bar, the gap between the address range size and the peak load size means fragmentation.



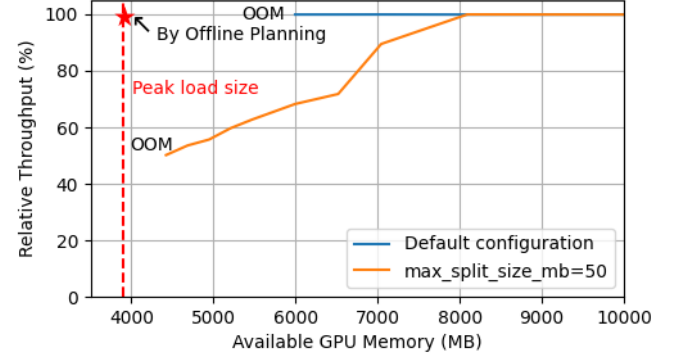
**Figure 2.** Intensive recomputation breaks the unimodal memory footprint of overall computation and generates complex allocation patterns.

Without recomputation, the memory consumption pattern is unimodal (increasing during the forward pass and decreasing during the backward pass), and most allocator algorithms exhibit little fragmentation at peak memory consumption. However, with recomputation enabled, allocations and releases of memory are interlaced throughout the computation, resulting in complex patterns.

Furthermore, given the inherent characteristics of recomputation, the PyTorch caching allocator can endure severe

fragmentation due to numerous small allocations interspersed between large allocations and releases of memory.

This fragmentation not only wastes GPU resources, but it also slows down the training process. This is because PyTorch’s CUDA caching allocator uses blocking `cudaMalloc` and `cudaFree` operations when it needs to release cached blocks to make space.

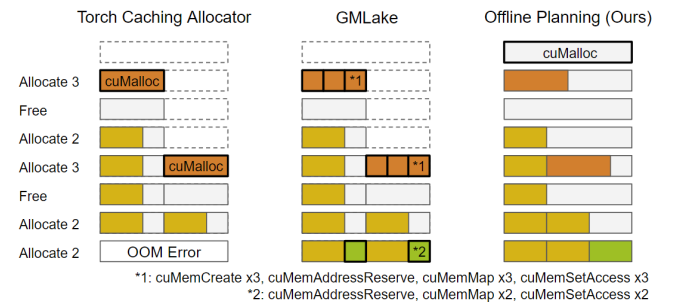


**Figure 3.** Training throughput against available GPU memory, using PyTorch caching allocator.

Figure 3 shows the performance degradation of the PyTorch caching allocator when the available memory is limited, measured on vision transformer (recomputation tuned) with batch size = 512. While the peak load size is 3.9 GB, the PyTorch caching allocator fails to run with <6 GB available memory due to fragmentation by caches. While this can be improved by configuring cache management (by setting `max_split_size_mb`), the throughput degrades to near 50 % due to the overheads of CUDA API calls. These overheads can be completely removed by offline allocation planning.

## 1.2 Offline Allocation Planning

Because memory allocation is periodic during neural network training, the allocation can be better planned once the allocation pattern is available.



**Figure 4.** Fragmentation by the PyTorch caching allocator (left) and defragmentation technique by GMLake [11] (middle) and offline planning (right). Each bold frame means that there are some blocking CUDA API calls on that memory segment.

Figure 4 depicts the idea of the offline planning. Compared to another approach (GMLake; detailed in Section 2.5), which runs online by stitching physical memory chunks into contiguous virtual ranges, the offline planning has limited applicability because it needs assumptions on the allocation patterns, but it enables maximal resource utilization without runtime overhead.

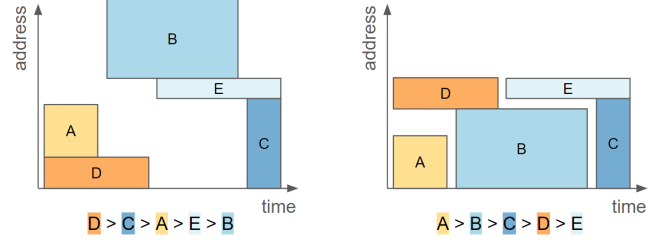
The problem behind the offline planning can be reinterpreted as follows: Consider a set of allocations  $A$ , where each allocation  $a \in A$  is configured to start at  $t_{1,a}$ , end at time  $t_{2,a}$ , and have size  $s_a$ ; our task is to assign a contiguous memory region of length  $s_a$  for each  $a \in A$  from a memory pool so that the two regions do not overlap when the lifetime intervals overlap (the formal definition is presented in Section 3.1). The memory address should be determined to minimize the size of the pool. This problem is known as the dynamic storage allocation (DSA) problem. The DSA problem is NP-hard [8], and thus optimizations by exact algorithms are too expensive. There are extensive research on polynomial-time approximation algorithms [6, 9, 10, 14]. However, these algorithms mainly focus on theoretical bounds, and their practical implementations often adopt heuristics such as first-fit or best-fit with coalescing [7, 16, 18]. According to Lamprkos et al., popular allocation algorithms, including the one used in glibc, which is based on the best-fit with coalescing strategy [16], can function as efficient DSA solvers [15]. PyTorch’s default GPU memory allocator, the CUDA caching allocator, also uses the best-fit with coalescing strategy [18]. We recognize the lack of recent research directly connected to DSA algorithms. Our work is primarily driven by the issue of memory fragmentation caused by recomputation, a problem that has gained prominence with the current advent of efficient recomputation algorithms like FastSA [12] and Rockmate [22]. Techniques to mitigate this issue are under active investigation. For instance, GMLake [11] seeks to tackle fragmentation through the application of virtual memory stitching.

For this offline dynamic storage allocation problem setting, for the allocation pattern without recomputation, we observed that the first-fit and best-fit strategy and the on-line torch allocator perform with almost no fragmentation (Figure 1). However, with recomputation, the first-fit and best-fit heuristics may produce around 10 % fragmentation, and we developed a heuristics based on simulated annealing to further reduce fragmentation.

### 1.3 Contributions of This Study

To determine an allocation plan with minimum fragmentation, we propose a heuristic algorithm for DSA, whose state is a permutation of allocations. Intuitively, this permutation represents the order of operations in the two-dimensional bin packing (Figure 5).

Given the operation order, we can construct an allocation plan in a greedy manner. We demonstrate that minimal



**Figure 5.** A permutation of allocations (rectangles) determines the allocation plan. The rectangles are placed at the lowest possible position according to the order.

fragmentation can be achieved with a specific operation order, which our heuristic seeks to identify by applying small updates to the permutation. The naive construction of the allocation plan, completed in a single step of simulated annealing under the specific operation order, takes  $O(N^2)$  time, where  $N$  represents the number of allocations. This makes the algorithm impractical when optimizing thousands of allocations.

To expedite this evaluation, we have developed an algorithm to evaluate  $N$  candidates in  $O(N \log N)$  time, allowing for per-candidate evaluation in  $O(\log N)$  amortized time. An added advantage of this algorithm is that it can take an arbitrary address assignment (such as one produced by the first-fit) as its initial state and further improve the assignment.

We have exhaustively evaluated our algorithm on both random allocation patterns and allocation patterns resulting from training popular neural network models with recomputation. When the number of allocations is fewer than 10,000, our algorithm can reduce fragmentation to under 0.4 % on average. To our best knowledge, our algorithm is the most powerful heuristic for the DSA problem which can handle thousands of allocations. In comparison, the average fragmentation caused by the PyTorch caching allocator, first-fit, and best-fit strategies were 29.4 %, 5.3 %, and 6.7 %, respectively.

We summarize our contributions as follows:

- To the best of our knowledge, this is the first DSA problem formulation as an optimization on the *topological* ordering among allocations. This formulation enables easier sampling of neighboring states in heuristic algorithms than maintaining rectangles in the two-dimensional bin packing problem or range-coloring formulation on interval/circular-arc graphs.
- We have developed a simulated annealing-based heuristic algorithm to the NP-hard DSA problem. It optimizes this topological ordering using non-trivial techniques such as the  $O(\log N)$  amortized time per-candidate evaluation.
- The initial solution in the simulated annealing can be any valid allocation plan and our algorithm has a

potential to improve it. Typically, we can start from the solution by the offline first-fit, which has relatively small fragmentation.

- Our algorithm has achieved more than 90 % reduction in fragmentation compared to commonly-used heuristics on allocation patterns from neural network training with recomputation, and on random patterns.

This algorithm has far-reaching applicability. It can be used to optimize GPU resources during training, deploy applications on embedded systems while maximizing resource usage, and optimize compile-time resources for various systems and architectures.

## 2 Related Work

This section introduces the techniques and algorithms used to reduce fragmentation from both engineering and theoretical perspectives.

### 2.1 Mechanism of PyTorch GPU Memory Allocation

In addition to the native memory management for GPU programs using `cudaMalloc` and `cudaFree`, most GPU frameworks use their own memory allocation mechanism because the native APIs are blocking and consequently significantly slow. PyTorch's CUDA caching allocator uses a set of blocks to manage the reserved GPU memory. This set of blocks is similar to the chunks in the `dldmalloc` [16], which were initially used in `glibc`. Allocation handling in its most basic form functions as follows: Given an allocation request, a block is determined using the best-fit method. If the found block is too large, the block is then split into two blocks, one of which is used to accommodate the data, and the other, which is empty, is returned to the set. When no block can handle the request, the caching allocator invokes `cudaMalloc` to reserve a new block. When the data are no longer required, the block becomes empty again, and two adjacent empty blocks coalesce to form a larger empty block. This implementation has low latency and generally performs well for typical neural network applications because there are many same-sized allocations during training or inference, and most allocations can be processed with the reserved empty blocks, without invoking `cudaMalloc`. Furthermore, because training and/or inference require repeated computation, the caching strategy performs exceptionally well until the resource is exhausted, at which point the cache will be destroyed by releasing some of the unsplit blocks using `cudaFree`. It should be noted that only unsplit blocks can be released by `cudaFree` and the partial memory in a split block cannot be released.

### 2.2 Existing Fragmentation Reduction Techniques in Caching Allocator

Fragmentation has always been a hot topic in memory optimization and PyTorch is not an exception. The PyTorch CUDA caching allocator has several options to configure

the algorithm to prevent OOM errors. As explained in the previous section, the existence of unused split blocks causes fragmentation in the CUDA caching allocator. To prevent splitting a large block for small allocations, PyTorch uses two block pools for small (up to 1 MB) and large allocations. Splitting large blocks is generally beneficial because it can reuse the reserved region, but it can also cause severe fragmentation. To prevent this, the caching allocator can be configured to not split larger blocks using a threshold. When such large blocks are no longer required, they can be released using `cudaFree` at any time; however, the synchronous `cudaFree` may reduce the execution throughput.

Furthermore, the caching allocator can be configured to round up each allocation size to the nearest power of two to prevent catastrophic fragmentation. However, in most workloads, this configuration may cause more fragmentation.

### 2.3 Offline Allocation for Neural Networks

When the allocation pattern is known in advance, either by simulation or profiling in larger environments, we can plan efficient allocation to minimize fragmentation, and this is exactly the scope of this paper. TensorFlow lite micro [7] can plan the memory layout before runtime. It sorts all allocations  $A$  by size  $s_a$  and then uses the first-fit method to determine the memory address for each allocation request [2], placing it in the first unused segments of sufficient size.

### 2.4 Allocation Algorithms and Fragmentation

The DSA problem, which is our interpretation of minimizing fragmentation, has been widely studied, and we introduce some theoretical findings here. Robson [19] showed that when the amount of memory in use is bounded and an optimal allocation strategy is used, the fragmentation size may increase logarithmically with respect to the number of allocations. A follow-up study demonstrated that the worst-case fragmentation for the first-fit strategy is also logarithmic, whereas the best-fit strategy may produce linearly increasing fragmentation [20]. However, Wilson et al. [21] reported that best-fit algorithms empirically produce less fragmentation than first-fit strategies.

Kierstead [14] introduced DSA as a variant of the graph coloring problem on interval graphs and provided a polynomial time six-approximation algorithm using the bound by weighted clique size. Numerous improvements have been made [9, 10, 17], and Confessore et al. [6] provided the two-approximation algorithm, which supports periodic allocation. Buchsbaum et al. [3] proposed a dynamic programming algorithm that boxes multiple rectangles into large rectangles to solve a simpler instance. However, practical allocators still outperform these algorithms as DSA solvers [15].

## 2.5 Memory Defragmentation by Virtual Memory Stitching

Recently, Guo et al. [11] proposed an allocator called GMLake to address the fragmentation during the training of deep neural networks. This method tackles the problem from another perspective by using CUDA's low-level virtual memory management APIs to stitch fragmented physical GPU memory chunks to create long virtual address ranges (Figure 4). This method serves as a drop-in replacement for PyTorch's CUDA caching allocator and is easy to adopt. However, when allocation patterns become highly irregular and GPU memory usage is continually oscillating near full capacity, as illustrated in Figure 2, this system may involve a significant amount of heavy operations (cuMemMap and cuMemSetAccess) and may potentially cause a severe degradation of training throughput. Considering the recent enhancements in GPU architecture's computational abilities, reducing such CPU overhead is especially important. In our method, these overheads are completely removed by planning the allocations offline.

## 3 Method

This section discusses the formalization of the DSA problem (section 3.1) and the implementation of our algorithm for solving it (section 3.2).

### 3.1 Problem Formalization

**Problem Input.** The task input is a set  $A$  that includes  $N$  allocation sequences. Each allocation involves one Alloc and one Free, which totals  $2N$  requests during the entire procedure. Each allocation  $a \in A$  allocates  $s_a$  bytes at time  $t_{1,a}$ , and frees at time  $t_{2,a}$ . Without losing generality, we can assume that each allocation and free request is queried at an integer time. To handle periodic allocations, let  $t_{\text{end}}$  be an integer that denotes the boundary time such that  $t_{1,a} < t_{\text{end}}$  and  $t_{2,a} < t_{\text{end}}$  for all allocations.

While common DSA formulations assume  $t_{1,a} \leq t_{2,a}$ , we exclude this constraint to handle allocations that are alive on the boundary times. More specifically, when  $t_{1,a} > t_{2,a}$ , it means that the memory is allocated at time  $t = t_{1,a} - t_{\text{end}} < 0$  in the previous iteration, and it remains alive on the boundary time  $t = 0$ , and is released at time  $t_{2,a}$ . We introduce the lifetime interval notation  $L_a$  as follows:

$$L_a := \begin{cases} [t_{1,a}, t_{2,a}) & (t_{1,a} \leq t_{2,a}) \\ [0, t_{2,a}) \cup [t_{1,a}, t_{\text{end}}) & (\text{otherwise}) \end{cases}$$

The load at time  $t$  is defined as the aggregate size of all allocations that are active immediately at time  $t$ . The peak memory size  $\text{Peak}_A$  during the procedure is defined as the maximum load at any time.

$$\text{Load}_A(t) := \sum_{a \in A} \{s_a | t \in L_a\}$$

$$\text{Peak}_A := \max_{0 \leq t < t_{\text{end}}} \text{Load}_A(t)$$

**Problem Output.** The task output involves address assignments  $x : A \rightarrow \mathbb{N}_{\geq 0}$ . The obtained address assignments must be memory-safe, indicating that no two allocations simultaneously share the same memory space. The condition for memory safety is defined as follows:

**Definition 3.1.** Each allocation  $a \in A$  allocates a contiguous memory range  $R_{x,a}$  defined as follows:

$$R_{x,a} = [x(a), x(a) + s_a)$$

An address assignment  $x$  is *memory-safe* if it satisfies the following condition:

$$\forall a, b \in A. (a \neq b \wedge L_a \cap L_b \neq \emptyset) \rightarrow R_{x,a} \cap R_{x,b} = \emptyset$$

The objective of this task is to find a memory-safe address assignment  $\{x_i\}$  that minimizes  $M_A(x)$ , the address range size required to complete all allocations, defined as follows:

$$M_A(x) := \max_{a \in A} \{x(a) + s_a\}$$

Furthermore, we introduce the concept of the *fragmentation ratio* as a metric to evaluate the effectiveness of the algorithm in finding address assignments. The fragmentation ratio is the proportion of the address range size relative to the peak load size. Formally, it is defined as follows:

$$F := \frac{M_A(x)}{\text{Peak}_A}$$

By evaluating the fragmentation ratio, we can evaluate and compare our algorithm's efficiency in memory allocation with other potential strategies.

### 3.2 About Our Approach

Our proposed approach uses *topological order* as an intermediate hint. Formally, our strategy  $S : A \mapsto x$  is defined as the combination of two subroutines,  $S_1 : A \mapsto P$  and  $S_2 : P \mapsto x$ , which use topologically ordered allocations  $P$ .

1.  $S_1$ : Determines an optimal *topologically ordered allocations*  $P$  of the provided allocations  $A$ .
2.  $S_2$ : Determines memory-safe address assignment  $x$  that minimize  $M_A(x)$  from  $P$ .

**Definition 3.2.**  $P$  is called *topologically ordered allocations*, or simply topological order, of address assignment  $x$  if the following conditions are satisfied:

- $\forall i > j. L_{P_i} \cap L_{P_j} \neq \emptyset \rightarrow x(P_i) \geq x(P_j) + s_{P_j}$ .

**Lemma 3.3.** If an address assignment  $x$  has at least one topological order  $P$ ,  $x$  is always memory safe.

*Proof.* From the condition of topological order,

$$\begin{aligned} \forall i \neq j. L_{P_i} \cap L_{P_j} \neq \emptyset \\ \rightarrow (x(P_i) \geq x(P_j) + s_{P_j} \vee x(P_j) \geq x(P_i) + s_{P_i}) \\ \rightarrow R_{x,P_i} \cap R_{x,P_j} = \emptyset \end{aligned}$$

□

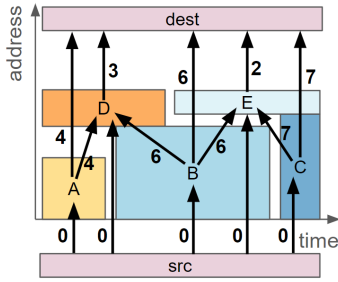
**Lemma 3.4.** *For any memory-safe address assignment  $x$ , there is at least one topological order.*

*Proof.* A permutation obtained by sorting  $A$  in ascending order of  $x(a)$  holds the condition of topological order.  $\square$

In particular, Lemma 3.4 demonstrates that at least one topological order exists that strictly minimizes  $M_A(x)$ . Conversely, for any valid topological order  $P$ ,  $\{x_{P_i}\}$  may not be sorted in ascending order.

To provide a clearer explanation, we begin the discussion with Algorithm  $S_2$ , which is simpler than  $S_1$ .

**3.2.1 Algorithm  $S_2$ .** In this section, we demonstrate that the exact solution of the address assignment that minimizes  $M_A(x)$  can be easily constructed for a given topological order.



**Figure 6.** With fixed topological ordering among allocations, the address range size corresponds to the longest path length of the DAG created as above.

**Lemma 3.5.** *For any allocation pattern  $A$  and permutation  $P$ , there exists at least one address assignment  $x$  that has a topological order  $P$ .*

*Proof.* The optimal address assignment  $x$  can be obtained by solving the following linear programming (LP):

$$\begin{aligned} &\text{minimize: } M \\ &\text{conditions: } \forall i. x(P_i) \geq 0 \\ &\quad \forall i. x(P_i) + s_{P_i} \leq M \\ &\quad \forall i > j. L_{P_i} \cap L_{P_j} \neq \emptyset \rightarrow x_{P_i} \geq x_{P_j} + s_{P_j} \end{aligned}$$

This LP is equivalent to the longest path problem for a directed acyclic graph (DAG) constructed as shown in Figure 6.

- DAG  $G = (V, E)$  has  $N + 2$  nodes.
  - $V$  consists of  $N$  nodes  $\{v_0, \dots, v_{N-1}\}$ , which correspond to each allocation pattern, the source  $v_{\text{src}}$ , and the destination  $v_{\text{dest}}$ .
  - $E$  is constructed using the following rules.
    - \* For any  $0 \leq i \leq N - 1$ , an edge from  $v_{\text{src}}$  to  $v_i$  of weight 0 exists.
    - \* For any  $0 \leq i \leq N - 1$ , an edge from  $v_i$  to  $v_{\text{dest}}$  of weight  $s_{P_i}$  exists.

- \* For any  $i > j$  such that  $L_{P_i} \cap L_{P_j} \neq \emptyset$ , an edge from  $v_j$  to  $v_i$  of weight  $s_{P_j}$  exists.
- \* No other edge exists.
- $M$  is the length of the longest path from  $v_{\text{src}}$  to  $v_{\text{dest}}$ .

$\square$

The length of the longest path of a DAG can be obtained in linear time with respect to the number of edges. Because the number of edges is at most  $O(N^2)$ , the following algorithm can be used to find the address assignment in  $O(N^2)$  time complexity.

**Algorithm 1** A  $O(N^2)$  algorithm for  $S_2$ .

```

1: for  $i = 0$  to  $N - 1$  do
2:    $x[P_i] \leftarrow 0$ 
3:   for  $j = 0$  to  $i - 1$  do
4:     if  $(L_i \cap L_j \neq \emptyset \text{ and } x[P_i] < x[P_j] + s_{P_j})$  then
5:        $x[P_i] \leftarrow x[P_j] + s_{P_j}$ 
6:     end if
7:   end for
8: end for
9: return  $x$ 

```

For performance improvements, Algorithm 2, which is equivalent to Algorithm 1, is developed. This algorithm returns the same results as Algorithm 1, but it has a shorter execution time.

**Algorithm 2** A  $O(N \log N)$  algorithm for  $S_2$ .

```

1:  $h \leftarrow$  zero array of length  $t_{\text{end}}$ 
2: for  $i = 0$  to  $N - 1$  do
3:    $x[P_i] \leftarrow \max_{t \in L_{P_i}} h[t]$ 
4:    $h[t \in L_{P_i}] \leftarrow x[P_i] + s_{P_i}$ 
5: end for
6: return  $x$ 

```

**Lemma 3.6.** *Algorithms 1 and 2 are equivalent.*

*Proof.* Consider the loop invariant for the array  $h$  as the inductive hypothesis. We assume that the array  $h$  maintains the following condition at the beginning of the  $i$ -th loop.

$$h[t] = \max \left( 0, \max_{0 \leq j < i \wedge t \in L_j} (x[P_j] + s_{P_j}) \right)$$

This condition must be maintained at the beginning of the 0-th loop because  $h$  is initialized as a zero array. Assuming the  $i$ -th loop, we can confirm that the following condition holds at the beginning of the  $i + 1$ -th loop. If  $t \in L_i$ , then  $x[P_i] + s_{P_i}$  must be greater than or equal to  $\max_{0 \leq j < i \wedge t \in L_j} (x[P_j] + s_{P_j})$ . Hence, the following conditions are maintained at the end of the  $i$ -th loop:

$$h[t] = \begin{cases} x[P_i] + s_{P_i} & \text{if } t \in L_i \\ \max_{0 \leq j < i \wedge t \in L_j} (x[P_j] + s_{P_j}) & \text{otherwise} \end{cases}$$

$$= \max_{0 \leq j \leq i \wedge t \in L_j} (x[P_j] + s_{P_j})$$

This validates the correctness of the loop invariant. The equivalence of  $x$  between Algorithms 1 and 2 is also established as follows:

$$x[P_i] = \max_{t \in L_{P_i}} h[t] = \max_{0 \leq j < i \wedge L_i \cap L_j \neq \emptyset} (x[P_j] + s_{P_j})$$

$$h[t] = \begin{cases} x[P_i] + s_{P_i} & \text{if } t \in L_i \\ \max_{0 \leq j < i \wedge t \in L_j} (x[P_j] + s_{P_j}) & \text{otherwise} \end{cases}$$

$$= \max_{0 \leq j \leq i \wedge t \in L_j} (x[P_j] + s_{P_j})$$

□

Lines 3 and 4 of Algorithm 2 can be performed in  $O(\log N)$  using a segment tree that supports range-assignment and range-max queries. Consequently,  $O(N \log N)$  is the total time complexity of Algorithm 2. Henceforth, the minimum size of peak memory under the condition of a given topological order  $P$  is denoted as  $M(P)$ .

**3.2.2 Algorithm  $S_1$ .** We now return to our discussion of  $S_1$ . It is practically challenging in terms of time to determine an exact solution for the topological order  $P$  that minimizes  $M(P)$ . Therefore, we suggest a simulated annealing-based heuristic algorithm (Algorithm 3) that can locate a  $M(P)$ -maximizing topological order.

---

**Algorithm 3** Naive simulated annealing for  $S_1$ .

---

```

1:  $P \leftarrow$  a topological order of  $x_0$ .
2: for  $Loop_{\max}$  times do
3:    $i \leftarrow \text{random}(\{0, \dots, N-1\})$ 
4:    $j \leftarrow \text{random}(\{0, \dots, N-1\})$ 
5:    $P' \leftarrow$  rotate  $P_i$  to  $j$ -th element of  $P$ .
6:   Find  $M(P')$  with Algorithm 2.
7:    $P \leftarrow P'$  probabilistically according to  $M(P')$ 
8: end for
9: return  $P$ 

```

---

This algorithm begins from  $x_0$ , which is obtained from a different simple algorithm that finds the address location (e.g., first-fit algorithm). The following simulated annealing method has a high probability of finding a better address location than  $x_0$ . In each loop iteration, one neighbor permutation  $P'$  that is constructed by randomly rotating  $P$  is evaluated in  $O(N \log N)$  time complexity; therefore, the total time complexity of Algorithm 3 is  $O(Loop_{\max} O(N \log N))$ .

---

**Algorithm 4** Faster simulated annealing for  $S_1$ .

---

```

1:  $P \leftarrow$  a topological order of  $x_0$ .
2: for  $Loop_{\max}$  times do
3:    $i \leftarrow \text{random}(\{0, \dots, N-1\})$ 
4:    $\tilde{P} \leftarrow$  remove the  $i$ -th element of  $P$ .
5:   Find  $M(\tilde{P})$  using Algorithm 2.
6:    $h_1 \leftarrow$  zeros array of length  $t_{\text{end}}$ 
7:   for  $j = 0$  to  $N-2$  do
8:      $x_1[j] \leftarrow \max_{t \in L_{P_i}} h_1[t]$ 
9:      $h_1[t \in L_{\tilde{P}_j}] \leftarrow \max_{t \in L_{\tilde{P}_j}} h_1[t] + s_{\tilde{P}_j}$ 
10:  end for
11:   $x_1[N-1] = \max_{t \in L_{P_i}} h_1[t]$ 
12:   $h_2 \leftarrow$  zeros array of length  $t_{\text{end}}$ 
13:  for  $j = N-1$  to  $1$  in descending order do
14:     $x_2[j] \leftarrow \max_{t \in L_{P_i}} h_2[t]$ 
15:     $h_2[t \in L_{\tilde{P}_j}] \leftarrow \max_{t \in L_{\tilde{P}_j}} h_2[t] + s_{\tilde{P}_j}$ 
16:  end for
17:   $x_2[0] = \max_{t \in L_{P_i}} h_2[t]$ 
18:  for  $j = 0$  to  $N-1$  do
19:     $P' \leftarrow$  insert  $P_i$  to  $j$ -th position of  $\tilde{P}$ 
20:     $X \leftarrow \max(\{M(\tilde{P}), x_1[j] + x_2[j] + s_{P_i}\})$ 
21:    assert  $M(P') = X$ 
22:     $P \leftarrow P'$  probabilistically according to  $X$ 
23:  end for
24: end for
25: return  $P$ 

```

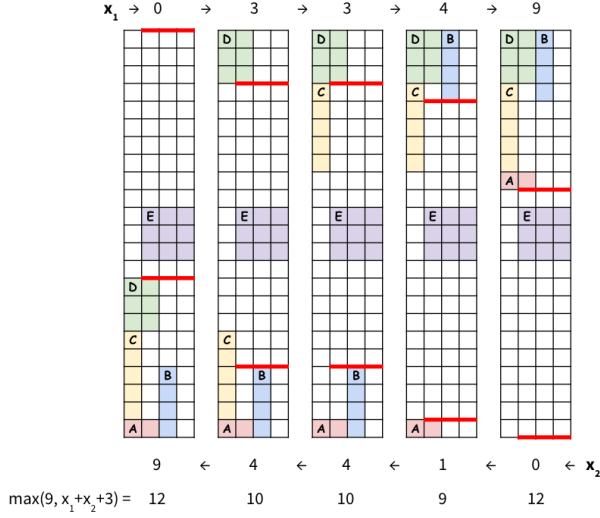
---

This algorithm takes too long to converge  $M(P)$  due to slow evaluation; therefore, we demonstrate an alternative faster algorithm that can evaluate  $N$  neighbors in  $O(N \log N)$  time complexity. Algorithm 4 evaluates the  $M(P')$  for all  $j = 0, \dots, N-1$  in Algorithm 3 without increasing the time complexity. To evaluate these  $N$  candidates, we first create a sequence  $\tilde{P}$  and remove the  $i$ -th allocation. Then, for each  $j = 0, \dots, N-1$ , we efficiently calculate the  $M(P')$ , where  $P'$  denotes the sequence obtained by inserting  $P_i$  as the  $j$ -th element of  $\tilde{P}$ , using the following two steps: up-sweep (shown in blue) and down-sweep (shown in violet).

Up-sweep and down-sweep compute arrays  $x_1$  and  $x_2$ , respectively.  $x_1[j]$  denotes the minimum address of allocation  $P_i$  when it is placed in the topological order  $[\tilde{P}_0, \dots, \tilde{P}_{j-1}, P_i]$ , and  $x_2[j]$  denotes the minimum address of allocation  $P_i$  when placed in the topological order  $[\tilde{P}_{N-2}, \dots, \tilde{P}_j, P_i]$ . As shown in Figure 7, using  $x_1$  and  $x_2$ , we can compute  $M(P')$  as  $\max(\{M(\tilde{P}), x_1[j] + x_2[j] + s_{P_i}\})$ .

To demonstrate the correctness of this algorithm, we introduce the following lemma of graph theory.

**Lemma 3.7.** Assuming that a weighted DAG  $G = (V, E)$  is given, where  $G$  has two vertices  $s$  and  $t$ , such that any  $v \in V$  is reachable from  $s$ , and  $t$  is reachable from any  $v \in V$ .  $d :$



**Figure 7.** Up-sweep and Down-sweep method to evaluate  $N$  candidates in  $O(N \log N)$  time. In the above case, the address range sizes for topological orders  $[A, B, C, D, E]$ ,  $[A, B, C, E, D]$ ,  $[A, B, E, C, D]$ ,  $[A, E, B, C, D]$ ,  $[E, A, B, C, D]$  are evaluated. Two arrays are used to maintain the address range sizes above and below  $E$ .

$V^2 \rightarrow \mathbb{R}$  is defined as the length of the longest path between any two given vertices. For any  $(v_1, v_2) \in E$  that has weight  $w$ ,  $d(s, t) = d(s, v_1) + w + d(v_2, t)$  holds.

**Lemma 3.8.** The assertion at line-21 in Algorithm 4 is always true.

*Proof.*  $x_1[j]$ ,  $x_2[j]$ , and  $M(\tilde{P})$  of our algorithm correspond to  $d(s, v_1)$ ,  $d(v_2, t)$ , and  $w$  of Lemma 3.7, respectively.  $\square$

Algorithm 4 enables the evaluation of  $N$  neighbors within  $O(N \log N)$  time complexity. This implies that the evaluation can be obtained within  $O(\log N)$ . This algorithm converges faster than Algorithm 3.

### 3.3 More Performance Tips

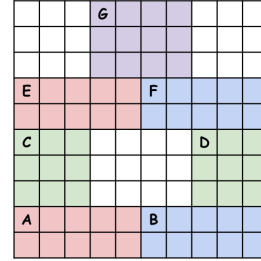
In this section, we discuss additional techniques for accelerating the rate of convergence in simulated annealing from the viewpoint of topological order.

**Worst-fit Heuristic.** Stochastically transitioning from a state where there are multiple choices of longest paths to a more optimal state is extremely challenging because shortening the longest path of the entire graph requires all longest paths to be reduced. Modifying line 20 in Algorithm 4 to  $X \leftarrow x_1[j] + x_2[j] + s_{P_i}$  can address this challenge. Empirical evidence suggests that this enhances the convergence speed.

The modification represents the length of the longest path that passes through the edge corresponding to the allocation  $P_i$ . Therefore, when this value is less than the longest path of

the entire graph, a spacious placement similar to the worst-fit strategy is evaluated to be preferable.

Using this value to evaluate states increases the frequency of transitions from states with numerous longest paths to a state with fewer longest paths, which encourages faster convergence toward a state with shorter longest paths.



**Figure 8.** Two topological ordering  $[A, C, B, E, D, F, G]$  and  $[A, C, E, B, D, F, G]$  both represent the above address assignment. While the former ordering can reach the optimal assignment  $[A, C, B, G, E, D, F]$  with a single rotation, the latter takes at least two rotations.

**Swapping Heuristic.** As illustrated in Figure 8, there are multiple topological orders that represent the same address assignment. However, their ease of transitioning to improved states can significantly differ. To address this issue, we introduced Algorithm 5.

---

#### Algorithm 5 $S_1$ with swapping heuristic

---

```

1:  $P \leftarrow$  a topological order of  $x_0$ .
2: for  $Loop_{max}$  times do
3:   Commands inside the loop of  $S_1$ 
4:    $h \leftarrow$  zeros array of length  $t_{end}$ 
5:    $x \leftarrow$  Algorithm 2
6:   while exists  $i$  such that  $(L_i \cap L_{i+1} = \emptyset$  and  $x[P_i] + s_{P_i} > x[P_{i+1}] + s_{P_{i+1}})$  do
7:      $P_i, P_{i+1} \leftarrow P_{i+1}, P_i$ 
8:   end while
9: end for
10: return  $P$ 

```

---

The swap heuristic in Algorithm 5 enhances the transition efficiency toward the optimal state without requiring alterations of address assignments. Consequently, even during annealing at lower temperatures, the likelihood of reaching a local optimum is diminished, leading to a faster convergence speed.

The while-loop operating between lines-6 and 8 exhibits a time complexity of  $O(NK)$ , where  $K$  represents the number of swaps. Leveraging Algorithm 6, these swaps can be executed with a time complexity of  $O(N + K)$  for improved efficiency.

**Algorithm 6**  $O(N + K)$  swapping algorithm

---

```

1:  $i \leftarrow 0$ 
2: while  $i < N - 1$  do
3:   if  $L_i \cap L_{i+1} = \emptyset$  and  $x[P_i] + s_{P_i} > x[P_{i+1}] + s_{P_{i+1}}$  then
4:      $P_i, P_{i+1} \leftarrow P_{i+1}, P_i$ 
5:      $i \leftarrow \max(\{0, i - 1\})$ 
6:   else
7:      $i \leftarrow i + 1$ 
8:   end if
9: end while

```

---

**3.4 Software Integration**

In this section, we shortly discuss the integration of our offline DSA algorithm into real applications with PyTorch. The allocator must handle not only the periodic allocation requests but also other general requests because there are often some preprocess (e.g., weight allocations before the training loop) and postprocess (e.g., releases on termination) outside the periodic requests. Our algorithm works well when the preprocess and postprocess requests consumes very little fraction of GPU memory and the periodic parts are predominant. In such cases, the allocator can handle preprocess and postprocess requests in a separated small pool, and plans periodic allocations using a large remaining pool. The scope of periodic allocations can be informed to the allocator using Python’s contextmanager. During the first iteration within the scope, the allocator calls `cudaMalloc` (or `cudaMallocManaged`) and `cudaFree` naively to process requests while tracing the requests. Once the periodic allocation pattern is obtained, it plans all allocations within the scope and then allocate requests within the memory pool. Therefore, while the first iteration may take time, there is no allocation overhead by `cudaMalloc` or `cudaFree` from the second iteration.

While GMLake [11] is easier to integrate because it has no assumption on the allocation patterns and works fully online, it involves many calls of `cuMemMap` and `cuMemSetAccess`, and potentially degrades the throughput. This is exceptionally problematic when the GPU memory usage is almost full and the allocator needs to remap many small physical memory chunks to make a contiguous long logical address ranges to handle large allocations.

Overall, while our method is offline and its applicability is limited, we believe it performs best when the allocations need to be highly tuned for specific workloads because it allows for near-full resource utilization and has no runtime overhead once allocations are planned.

**4 Experiments**

This section presents the experimental results of our algorithm using randomly generated data and allocation patterns obtained from neural network training using recomputation.

**Table 1.** Vision and text models

| Short Name   | Full Model Name             | $N$  |
|--------------|-----------------------------|------|
| resnet       | resnet50                    | 2722 |
| vit          | vit_small_patch16_224       | 2944 |
| mobilenet    | mobilenetv3_large_100       | 2981 |
| beit         | beit_base_patch16_224       | 4844 |
| deit         | deit3_base_patch16_224      | 5116 |
| convnext     | convnext_tiny               | 7608 |
| efficientnet | efficientnet_b0             | 9348 |
| distilbert   | distilbert-base-uncased     | 2141 |
| bert         | bert-base-uncased           | 4164 |
| electra      | electra-small-discriminator | 4298 |
| albert       | albert-base-v2              | 5342 |

All experiments were conducted on Intel(R) Xeon(R) Gold 6154 CPU @ 3.00 GHz. The implementation was written in C++ and compiled using GCC 9.4.0 with the O2 option.

**4.1 Data Preparation**

We collected famous vision and text models from timm (PyTorch image models) and transformers from Hugging Face. The models were configured for training for classification tasks, with recomputation. Applying recomputation effectively reduces the peak load size of the training, but it may cause significant memory fragmentation. We used the state-of-the-art algorithm FastSA [12] recomputation solver, further optimized by graph contraction [13], which is applicable to any computation graph. From the optimized computational graph, we extracted the lifetime intervals and sizes for each tensor. Batch sizes were set to 512 and 128 for the vision and text models, respectively. The sequence length was set to 512 for the text models.

For each model, the number of allocations is summarized in Table 1. The number of allocations was between 2,141 and 9,348.

We also tested our algorithm using random data. For random data, we generated five instances with different seeds. Each instance contained 1,000 allocations, where the start time is a random integer between 0 and 1,000, and both the duration and allocation size are random integers between 1 and 100.

**4.2 Competitor Algorithms**

We compared our algorithm with two widely-used offline strategies and the online strategy used in PyTorch caching allocator. The offline variants of the first-fit and best-fit algorithms do not handle each allocation in the order of requests and performs much better than their online counterparts.

**Offline Best-fit Strategy.** The best-fit strategy is widely known for its superior performance among offline DSA algorithms. The specific details of our implementation used for our experiments are follows:

- Sort all allocations in  $A$  in descending order of  $s_a$ . For two allocations with the same  $s_a$ , ties are broken so that the one with smaller  $t_{1,a}$  comes first.
- Determine the address of each allocation in the sorted order. For each allocation  $a$ , find the smallest free contiguous address range  $[start, end)$  that can accommodate the allocation size  $s_a$ . The assignment is  $x(a) = start$ .

**Offline First-fit Strategy.** The first-fit strategy is also known as a good offline DSA algorithm; however, in our experiments, it outperformed the best-fit strategy. The specific details of our experimental implementation are as follows:

- Sort all allocations  $A$  in the same way as the best-fit strategy.
- Determine the address of each allocation in the sorted order. For each allocation  $a$ , find the free address range  $[start, end)$  with the smallest start that can accommodate  $a$ , and let  $x(a) = start$ .

**PyTorch CUDA Caching Allocator.** PyTorch incorporates its own caching allocator for GPUs to reduce the frequency of blocking function calls such as `cudaMalloc` and `cudaFree`. This caching allocator optimizes memory allocation and free operations, significantly improving the overall performance of GPU operations within the PyTorch framework. The PyTorch allocator uses a highly efficient online DSA algorithm. Our study used this algorithm as a benchmark for comparison.

### 4.3 Hyperparameters for Simulated Annealing

In Algorithm 4, we probabilistically updated the permutation  $P$ , even when the objective  $M(P)$  became slightly worse, to avoid local minima and explore a wider solution space. The temperature determines the probability. In our experiments,  $\tau_i$ , the temperature in the  $i$ -th loop of the simulated annealing, is set as follows:

$$\tau_i = e^{(\log \tau_s - \log \tau_e) i / Loop_{max} + \log(\tau_s)},$$

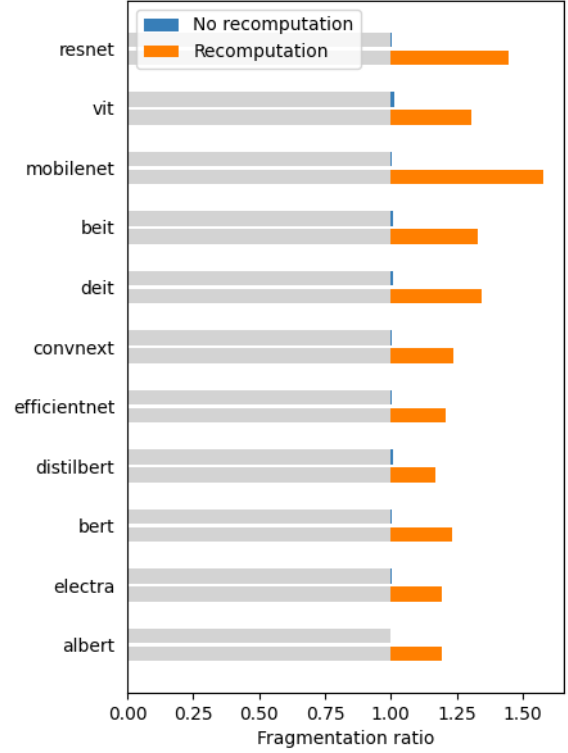
where  $\tau_s = Peak_A/300$  and  $\tau_e = \min(100, \tau_s/1000)$  denote the starting and ending temperatures, respectively. In the  $i$ -th loop, the permutation  $P$  is updated with  $P'$  using the following probability:

$$\min(1, e^{\frac{M(P) - M(P')}{\tau_i}})$$

The number of loops in the simulated annealing was set to 1,000,000 for all instances. An allocation with zero fragmentation will also end the loop.

## 4.4 Results

**4.4.1 Fragmentation Increase by Recomputation.** First, we empirically demonstrate that intensive recomputation may cause severe fragmentation when using the default PyTorch caching allocator. As shown in Figure 9, when the load size is reduced using recomputation, the fragmentation ratio

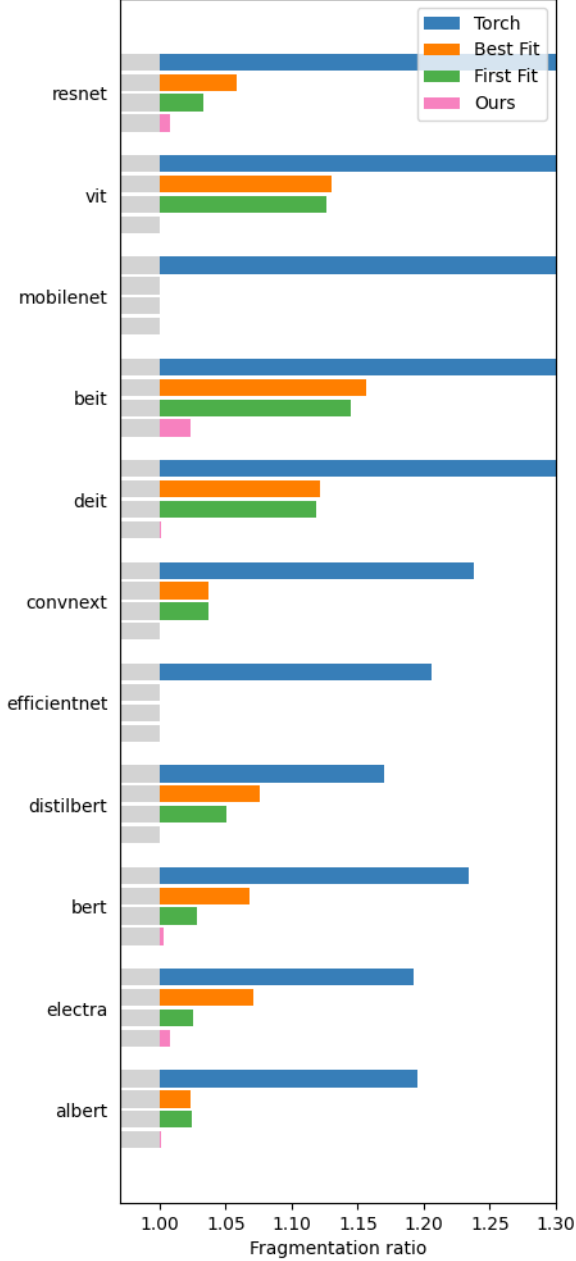


**Figure 9.** When recomputation is enabled, PyTorch caching allocator wastes large fraction of memory due to fragmentation.

increases. Without recomputation, there is 0.53 % fragmentation compared with the original peak load size. However, with recomputation, there is 29.5 % fragmentation, and for the mobilenet case, the ratio of fragmentation is 57.7 %, meaning that 36.6 % of the memory consumed by the allocator is not used.

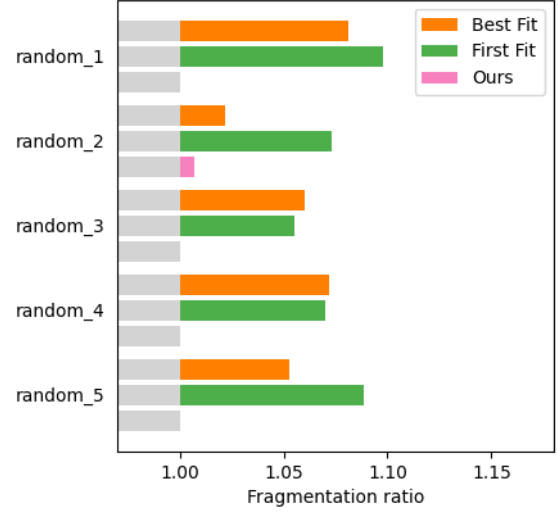
**4.4.2 Comparison of Offline Dynamic Storage Allocation Algorithms.** To evaluate the fragmentation reduction achieved using our algorithm, we compared its performance with three heuristics: torch caching allocator, best-fit, and first-fit. In our simulated annealing, we optimized the first-fit results as the initial solution. Simulated annealing was performed using 30 different seeds, and the best solution was selected. Figure 10 shows the results. First-fit and best-fit strategies can reduce fragmentation to a smaller extent than the torch caching allocator, and for some models, such as mobilenet and efficientnet, they can even completely remove fragmentation. However, they may still produce fragmentation up to 15.6 % and 14.5 % on beit. Our algorithm averagely reduced fragmentation to 0.4 %, with 2.4 % on beit being the worst case.

**4.4.3 Comparison on Random Data.** To demonstrate the performance of our algorithm on a wider range of data, we compared our algorithm with first-fit and best-fit heuristics on randomly generated data. The results are summarized



**Figure 10.** Fragmentation reduction using our algorithm.

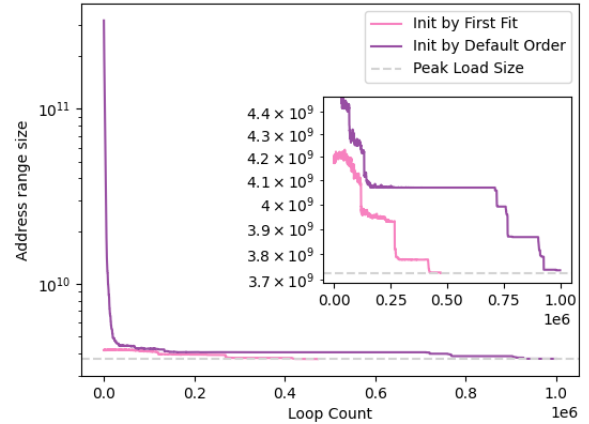
in Figure 11. We generated five random allocation patterns comprising 1,000 allocations with different seeds. Because the number of loops in the simulated annealing is sufficient for the size of the problem, we used the default seed. On average, the first-fit and best-fit strategy produced 5.76 % and 7.69 % fragmentation, respectively, whereas our algorithm could remove it, except for instance 2. Although the offline first-fit method achieved relatively low fragmentation ratios in Figure 10, we believe that the significance of our algorithm continues to augment, considering the recent dramatic improvements in recomputation algorithms.



**Figure 11.** Comparison on random allocations.

#### 4.5 Limitation of Our Algorithm

This section discusses the limitations of our algorithm. Our method, which is a simulated annealing algorithm, requires sufficiently large loops for convergence. The number of necessary loops typically depends on the number of allocations. Furthermore, to expedite convergence, we selected a good initial solution. We observed that the offline first-fit method yields satisfactory (or even optimal) solutions.



**Figure 12.** Convergence with different initial states on vit\_small\_patch16\_224.

Figure 12 shows the difference in convergence speed between the two initial states of the simulated annealing, the first-fit solution, and the default order (i.e., the order in which requests occur). After 1,000,000 simulated annealing loops, we could not obtain the optimal solution for all 30 different seeds if the initial state was the default ordering. In order to avoid local minima, it is helpful to have a good initial solution that is not far from the optimal one.

Furthermore, we observed that 1,000,000 loops may not be sufficient for larger instances exceeding 10,000 allocations, even when first-fit initial solutions are used.

**Table 2.** Large instances

| Short Name | Full Model Name         | $N$   |
|------------|-------------------------|-------|
| xcit       | xcit_tiny_12_p16_224    | 10620 |
| vit_large  | vit_large_patch16_224   | 10954 |
| convnextv2 | convnextv2_large        | 11921 |
| coatnet    | coatnet_2_rw_224        | 16986 |
| eva        | eva02_large_patch14_224 | 18847 |
| opt        | opt-6.7b                | 11013 |
| bloom      | bloom-3b                | 11605 |
| gpt-neo    | gpt-neo-2.7B            | 16101 |
| llama      | llama-7b                | 26814 |

To test this limitation, we used our algorithm on language models and large vision transformers listed in Table 2 whose allocation sequences contain more than 10,000 items. For the models in Table 2, the batch size and sequence length were set to 8 and 2,048, respectively.

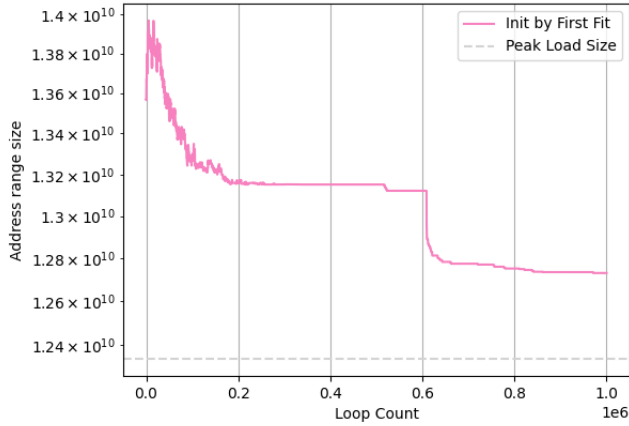
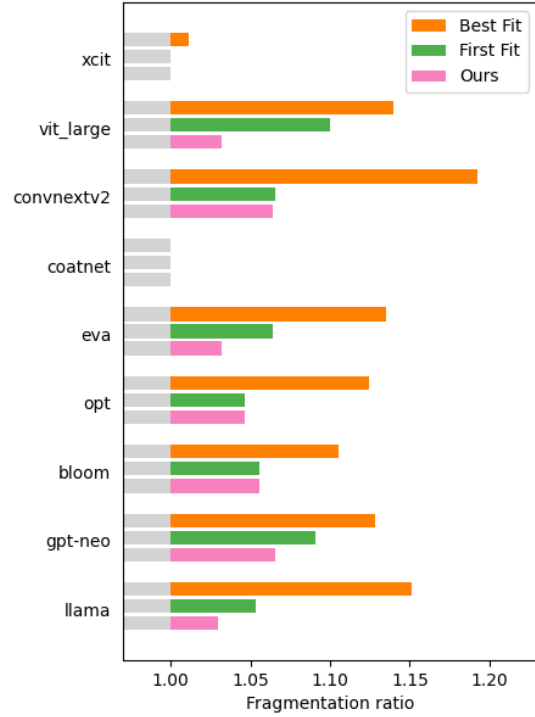
**Figure 13.** Slow convergence on large vit\_large.

Figure 13 shows the slow convergence of a large instance, vit\_large\_patch16\_224 with 10,954 allocations. With 1,000,000 iterations, the objective of simulated annealing is still gradually decreasing and does not fully converge. When the instance is significantly large, the length of time each simulated annealing loop takes increases. For example, the optimization on distilbert (2,141 allocations) was completed within 9 min, but for llama (26,814 allocations), it took approximately 90 min.

Figure 14 summarizes the performance of the proposed method for large instances. Our algorithm reduced fragmentation for all instances except for bloom; however, the improvement over the initial solution, i.e., first-fit, was smaller than that observed in the small cases.

In summary, although our algorithm is still applicable to larger instances with 10,000 allocations, its performance may be limited owing to insufficient convergence. Fortunately, these large instances are primarily transformer models with repeated structures. Although the FastSA recomputation

**Figure 14.** Performance comparison on large instances

planner optimizes the computational graph globally and produces complex recomputation plans, other recomputation algorithms such as Rockmate [22] target such repeated structures and produce structured recomputation plans. Using such recomputation planners might help decompose large allocation plans into smaller repeated sequences.

## 5 Conclusion

This study proposes a simulated annealing algorithm to solve the offline DSA problem. We defined the problem as finding an optimal permutation of allocations from which addresses can be assigned with minimum fragmentation. To expedite the simulated annealing and efficiently manage addresses, we used the segment-tree data structure in Algorithm 2. We introduced the up-sweep and down-sweep techniques in Algorithm 4, which can evaluate  $N$  neighbors in  $O(N \log N)$  time, achieving  $O(\log N)$  evaluation per candidate.

Our experiments demonstrated that our algorithm performs well, particularly on instances with at most 10,000 allocations. Although its scalability begins to taper off over tens of thousands of allocations due to insufficient convergence, it still scales better than the existing best heuristic [3] by dynamic programming. Furthermore, it reduced fragmentation to 0.4 % compared with the first-fit and best-fit algorithms, which produced fragmentation up to 15.6 %. Our algorithm demonstrated good performance for random allocations; however, it had poor convergence when applied to larger instances over tens of thousands of allocations or when the initial states were far from optimal solutions.

## References

- [1] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. 2021. Efficient combination of rematerialization and offloading for training dnn. *Advances in Neural Information Processing Systems* 34 (2021), 23844–23857.
- [2] Richard P. Brent. 1989. Efficient implementation of the first-fit strategy for dynamic storage allocation. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 11, 3 (1989), 388–403.
- [3] Adam L Buchsbaum, Howard Karloff, Claire Kenyon, Nick Reingold, and Mikkel Thorup. 2003. OPT versus LOAD in dynamic storage allocation. In *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*. 556–564.
- [4] Ayan Chakrabarti and Benjamin Moseley. 2019. Backprop with approximate activations for memory-efficient network training. *Advances in Neural Information Processing Systems* 32 (2019).
- [5] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [6] Giuseppe Confessore, Paolo Dell’Olmo, and Stefano Giordani. 2001. An approximation result for a periodic allocation problem. *Discrete Applied Mathematics* 112, 1-3 (2001), 53–72.
- [7] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Tiezhen Wang, et al. 2021. Tensorflow lite micro: Embedded machine learning for tinyml systems. *Proceedings of Machine Learning and Systems* 3 (2021), 800–811.
- [8] Michael R Garey and David S Johnson. 1979. Computers and intractability. *A Guide to the Theory of NP-Completeness* (1979).
- [9] Jordan Gergov. 1996. Approximation algorithms for dynamic storage allocation. In *European Symposium on Algorithms*. Springer, 52–61.
- [10] Jordan Gergov. 1999. Algorithms for compile-time memory optimization. In *Proceedings of the tenth annual ACM-SIAM symposium on Discrete algorithms*. 907–908.
- [11] Cong Guo, Rui Zhang, Jiale Xu, Jingwen Leng, Zihan Liu, Ziyu Huang, Minyi Guo, Hao Wu, Shouren Zhao, Junping Zhao, et al. 2024. GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 450–466.
- [12] Akifumi Imanishi, Zijian Xu, Masayuki Takagi, Sixue Wang, and Emilio Castillo. 2023. A fast heuristic to optimize time-space tradeoff for large models. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- [13] Yuya Kadono, Akifumi Imanishi, and Zijian Xu. 2023. Improve Recomputation Algorithm by Graph Contraction Preprocessing. [https://tech.preferred.jp/ja/blog/improve\\_recomputation\\_algorithm\\_by\\_graph\\_contraction\\_preprocessing/](https://tech.preferred.jp/ja/blog/improve_recomputation_algorithm_by_graph_contraction_preprocessing/).
- [14] Hal A Kierstead. 1991. A polynomial time approximation algorithm for dynamic storage allocation. *Discrete Mathematics* 88, 2-3 (1991), 231–237.
- [15] Christos Panagiotis Lamprakos, Sotirios Xydis, Francky Catthoor, and Dimitrios Soudris. 2023. The Unexpected Efficiency of Bin Packing Algorithms for Dynamic Storage Allocation in the Wild: An Intellectual Abstract. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on Memory Management*. 58–70.
- [16] Doug Lea and Wolfram Gloger. 1996. A memory allocator.
- [17] NS Narayanaswamy. 2004. Dynamic storage allocation and on-line colouring interval graphs. In *International Computing and Combinatorics Conference*. Springer, 329–338.
- [18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [19] John Michael Robson. 1971. An estimate of the store size necessary for dynamic storage allocation. *Journal of the ACM (JACM)* 18, 3 (1971), 416–423.
- [20] John M Robson. 1977. Worst case fragmentation of first fit and best fit storage allocation strategies. *Comput. J.* 20, 3 (1977), 242–244.
- [21] Paul R Wilson, Mark S Johnstone, Michael Neely, and David Boles. 1995. Dynamic storage allocation: A survey and critical review. In *Memory Management: International Workshop IWMM 95 Kinross, UK, September 27–29, 1995 Proceedings*. Springer, 1–116.
- [22] Xunyi Zhao, Théotime Le Hellard, Lionel Eyraud-Dubois, Julia Gusak, and Olivier Beaumont. 2023. Rockmate: an efficient, fast, automatic and generic tool for re-materialization in pytorch. In *International Conference on Machine Learning*. PMLR, 42018–42045.

Received 2024-03-22; accepted 2024-05-10